Project : SPI Slave with Single Port RAM

NAME :

1.Abdalrahman Taha Mahmed

2.Mohamed Mansour Osman

3.Mohammed Anwar Abd allatif

RAM CODE :

RAM.v

```
module  RAM #( parameter MEM_DEPTH = 256,
               parameter ADDR_SIZE = 8)
             ( output reg [7:0] dout,
               output reg tx_valid,
               input [9:0] din,
               input clk,rst_n,rx_valid);
    reg [ADDR_SIZE-1:0] ram [MEM_DEPTH-1:0] ;
    reg [ADDR_SIZE-1:0] w_addr,r_addr;
    always @(posedge clk) begin
        if (!rst_n) begin
            dout <= 0;
            tx_valid <= 0;
        end
        else if (rx_valid) begin
            if (din[9:8] == 2'b00) begin
                w_addr <= din[7:0];
                tx_valid <= 0;
            end
            else if (din[9:8] == 2'b01) begin
                ram[w_addr] <= din[7:0];
                tx_valid <= 0;
            end
            else if (din[9:8] == 2'b10) begin
                r_addr <= din[7:0];
                tx_valid <= 0;
            end
            else if (din[9:8] == 2'b11) begin
                dout <= ram[r_addr];
                tx_valid <= 1;
            end
        end
    end
endmodule
```

SLAVE CODE :

slave.v

```
module SPI (
    input clk, rst_n,
    input MOSI, SS_n,
    input tx_valid,
    input [7:0]tx_data,

    output reg MISO,
    output reg rx_valid,
    output reg [9:0]rx_data
);
    parameter IDLE = 3'b000;
    parameter CHK_CMD = 3'b001;
    parameter WRITE = 3'b010;
    parameter READ_ADD = 3'b011;
    parameter READ_DATA = 3'b100;

    reg read_trans;
    reg [4:0] cs, ns;

// Next state
    always @(*) begin
        case (cs)
            IDLE:
                if (SS_n)
                    ns = IDLE;
                else
                    ns = CHK_CMD;
            CHK_CMD:
                if (SS_n)
                    ns = IDLE;
                else if (~SS_n && ~MOSI)
                    ns = WRITE;
                else if (~SS_n && MOSI && ~read_trans)
                    ns = READ_ADD;
                else
                    ns = READ_DATA;
            WRITE:
                if (~SS_n)
                    ns = WRITE;
                else
                    ns = IDLE;
            READ_ADD:
                if (~SS_n)
                    ns = READ_ADD;
                else
                    ns = IDLE;
```

```
            READ_DATA:
                if (~SS_n)
                    ns = READ_DATA;
                else
                    ns = IDLE;
            default: ns = IDLE;
        endcase
    end

// State memory
    always @(posedge clk) begin
        if (~rst_n) begin
            cs <= IDLE;
        end else
            cs <= ns;
    end

// Output
    reg [4:0] counter;
    always @(posedge clk) begin
        if (~rst_n) begin
            counter <= 0;
            rx_data <= 0;
            MISO <= 0;
            rx_valid <= 0;
            read_trans = 0;
        end
```

```
        else if (cs != IDLE && cs != CHK_CMD) begin
                if(cs == READ_ADD)
                    read_trans = 1;
                else if(cs == READ_DATA)
                    read_trans = 0;

                if (counter<=9)
                    rx_data = {MOSI, rx_data[9:1]};

                if (counter == 9)
                    rx_valid = 1;
                else if (counter == 11 || counter == 0)
                    rx_valid <= 0;

                if (tx_valid) begin
                    if (counter==11) MISO <= tx_data[0];
                    else if (counter==12) MISO <= tx_data[1];
                    else if (counter==13) MISO <= tx_data[2];
                    else if (counter==14) MISO <= tx_data[3];
                    else if (counter==15) MISO <= tx_data[4];
                    else if (counter==16) MISO <= tx_data[5];
                    else if (counter==17) MISO <= tx_data[6];
                    else if (counter==18) MISO <= tx_data[7];
                end

                counter <= counter + 1;
        end else counter = 0;
    end
endmodule
```

TOP MODULE CODE :

top_module.v

```
module top_module (
    input clk, rst_n,
    input MOSI, SS_n,

    output MISO
);

    wire tx_valid, rx_valid;
    wire [9:0]rx_data;
    wire [7:0]tx_data;

    SPI spiBlock(
    .clk(clk), .rst_n(rst_n),
    .MOSI(MOSI), .SS_n(SS_n),

    .tx_valid(tx_valid),
    .tx_data(tx_data),

    .MISO(MISO),
    .rx_valid(rx_valid),  .rx_data(rx_data)
    );

    RAM #(
        .MEM_DEPTH(256),
        .ADDR_SIZE(8)
    )ramBlock(
        .clk(clk), .rst_n(rst_n),
        .din(rx_data),
        .rx_valid(rx_valid),
        .dout(tx_data),
        .tx_valid(tx_valid)
    );
endmodule
```

TOP MODULE TEST BENCH :

top_module_tb.v

```
module top_module_tb ();
    reg clk, rst_n;
    reg MOSI, SS_n;

    wire MISO;

    top_module ins(
        .clk(clk), .rst_n(rst_n),
        .MOSI(MOSI), .SS_n(SS_n),
        .MISO(MISO)
    );

    parameter READ = 1;
    parameter WRITE = 0;

    reg [9:0] MOSI_array;
    reg [7:0] MISO_array;
    integer i;
    integer j;
    reg [7:0] chk_data [255:0];

    initial begin
        clk = 0;
        forever #1 clk=~clk;
    end

    initial begin
        $readmemb ("ram.dat", ins.ramBlock.ram);
        rst_chk;
        j=0;
        repeat(256) begin
        // Write Address
            select_state(WRITE);

            i=0;
            MOSI_array[9:8] = 2'b00;
            MOSI_array[7:0] = j;
            repeat(10) begin
                MOSI = MOSI_array[i];
                @(negedge clk);
                i=i+1;
            end

            SS_n = 1;
            @(negedge clk);

```

```
        // Write Data
            select_state(WRITE);

            i=0;
            MOSI_input_write(2'b01);
            chk_data[j] = MOSI_array[7:0];

            SS_n = 1;
            @(negedge clk);
            j=j+1;
        end
        j=0;
        repeat(256) begin
        // Read Address
            select_state(READ);

            i=0;
            MOSI_array[9:8] = 2'b10;
            MOSI_array[7:0] = j;
            repeat(10) begin
                MOSI = MOSI_array[i];
                @(negedge clk);
                i=i+1;
            end

            SS_n = 1;
            @(negedge clk);

        // Read Data
            select_state(READ);

            i=0;
            MOSI_input_read({2'b11, 8'b00000000});
            @(negedge clk);

            i=0;
            repeat(8) begin
                @(negedge clk);
                MISO_array[i] = MISO;
                i=i+1;
            end
            if (MISO_array != chk_data[j])
                $display("%t: Error", $time);

```

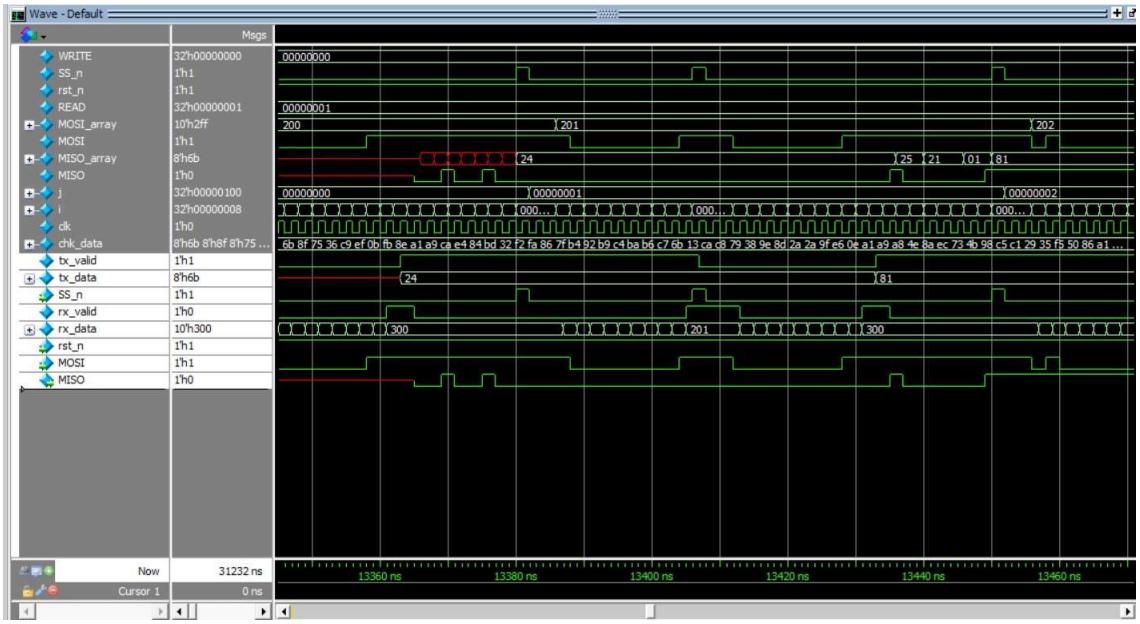
```
            SS_n = 1;
            @(negedge clk);
            j=j+1;
        end
        $stop;
    end

    task rst_chk;
        begin
        rst_n = 0;
        @(negedge clk);
        if (MISO)
            $display("%t: Error - rst",$time);
        rst_n = 1;
        end
    endtask

    task select_state(input STATE);
        begin
        SS_n = 0;
        @(negedge clk);
        MOSI = STATE;
        @(negedge clk);
        end
    endtask

    task MOSI_input_write(input [1:0]OPERATION);
        begin
        MOSI_array[9:8] = OPERATION;
        MOSI_array[7:0] = $random;
        repeat(10) begin
            MOSI = MOSI_array[i];
            @(negedge clk);
            i=i+1;
        end
        end
    endtask

    task MOSI_input_read(input [9:0] MOSI_IN);
    begin
        repeat(10) begin
            MOSI = MOSI_IN[i];
            @(negedge clk);
            i=i+1;
        end
    end
    endtask
endmodule
```

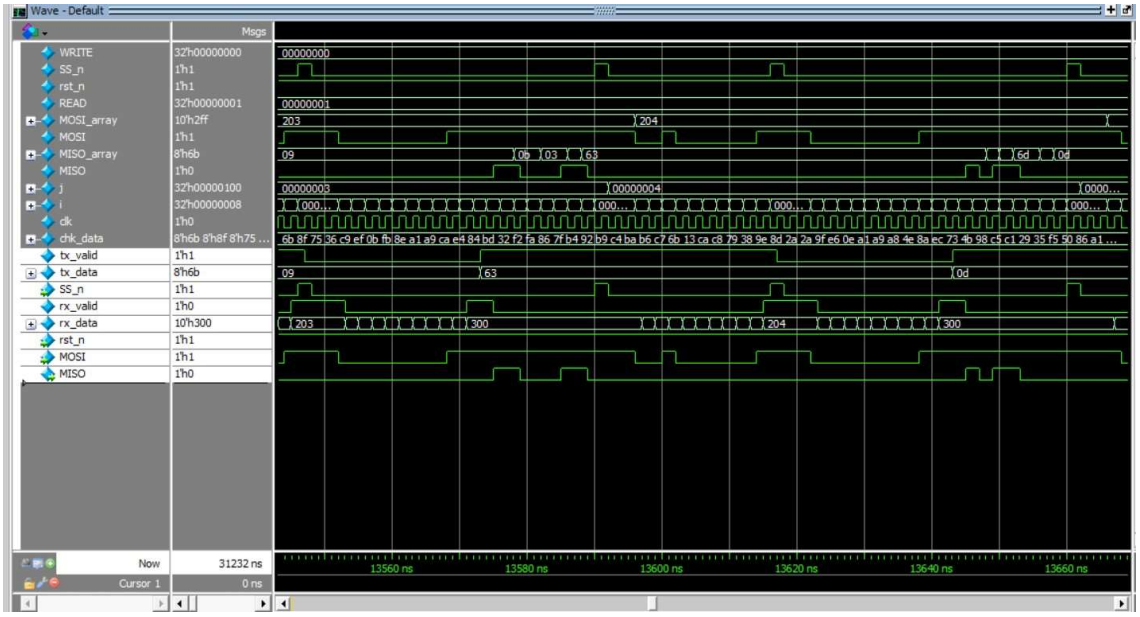
DO FILE:

SPI.do

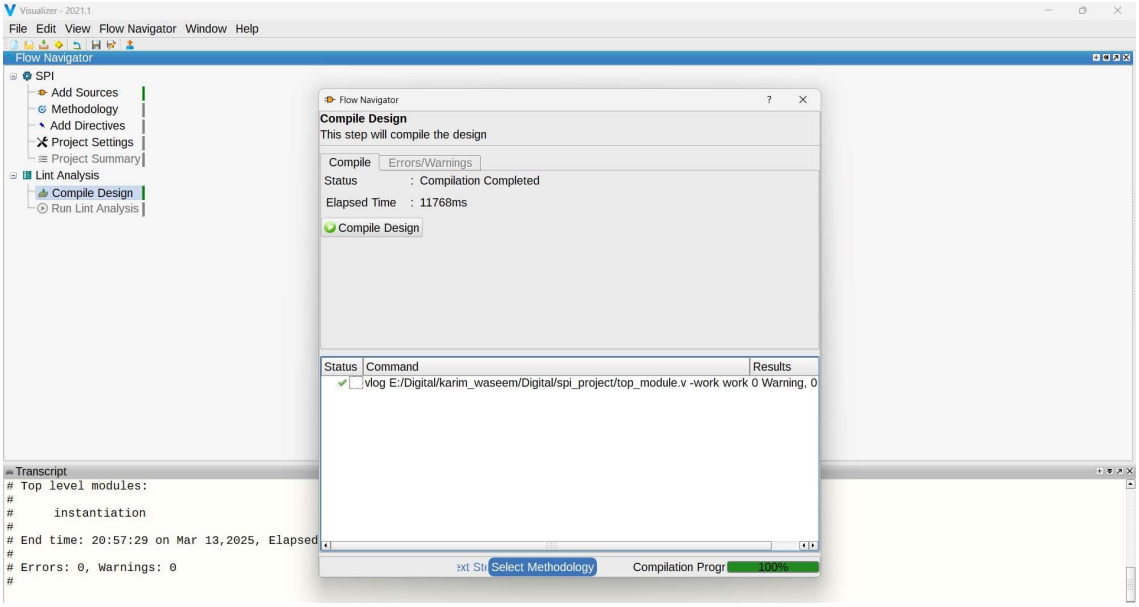
```
vlib work
vlog slave.v RAM.v top_module.v
vsim -voptargs=+acc work.top_module_tb
add wave*
run -all
```

WAVE SIMULATION :

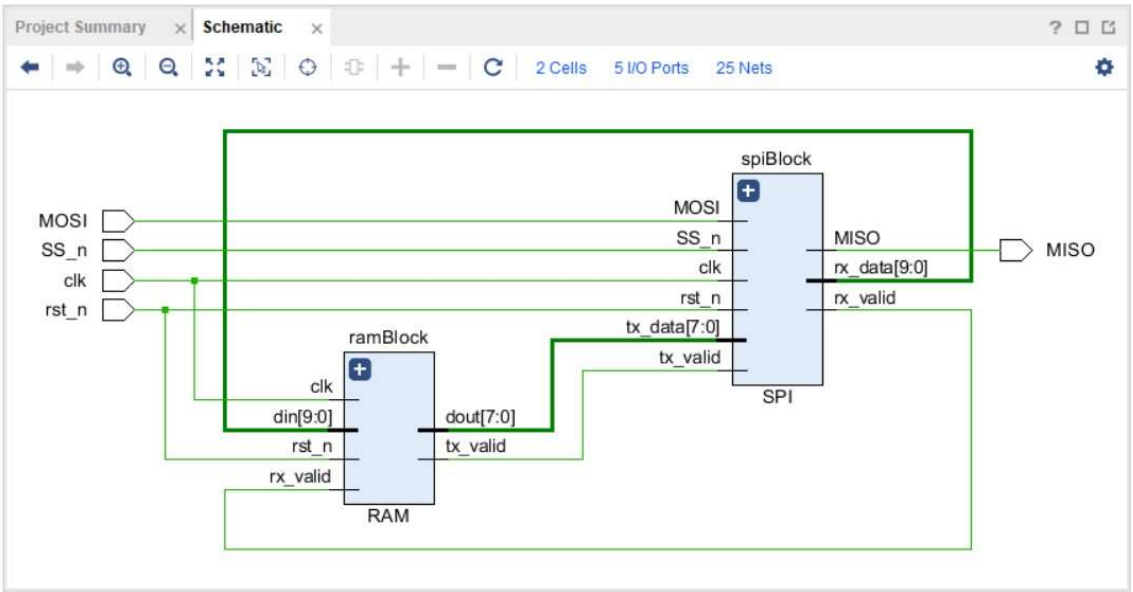




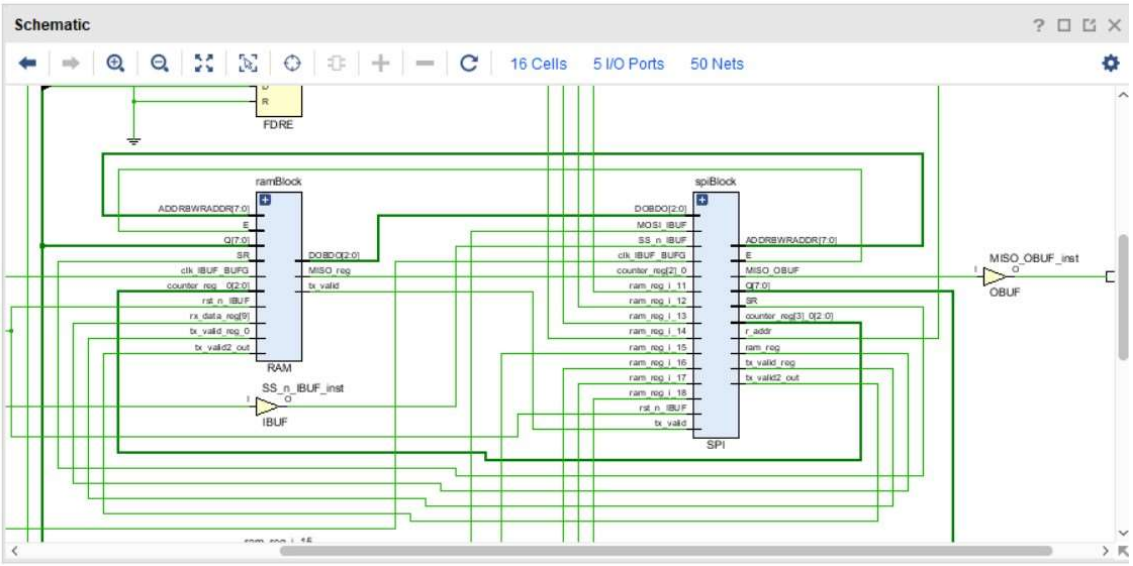
LINTING TOOL COMPILE :



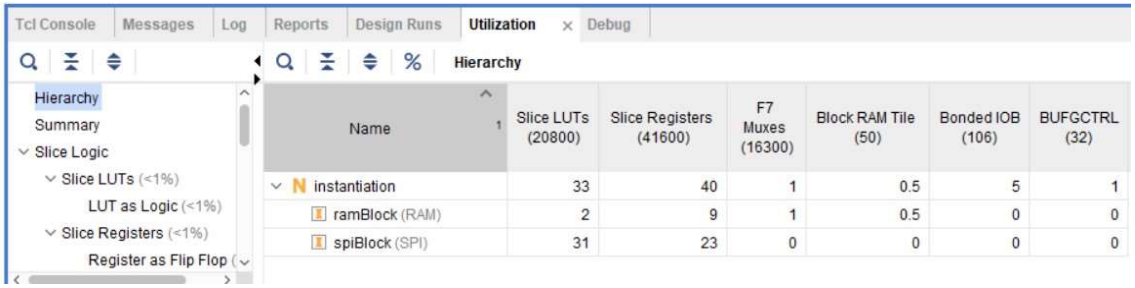
VIVADO ANALYSIS:



VIVADO SYNTHESIS :

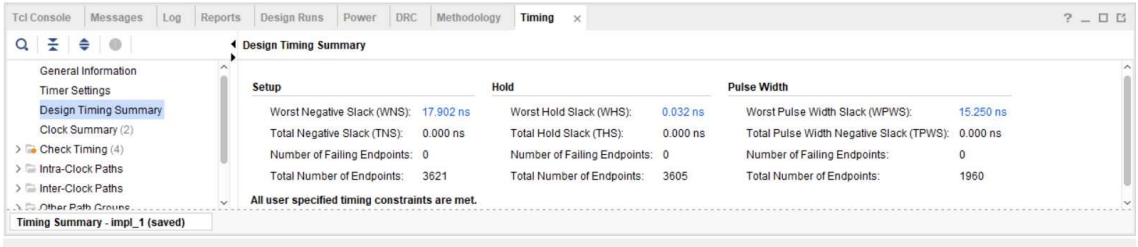
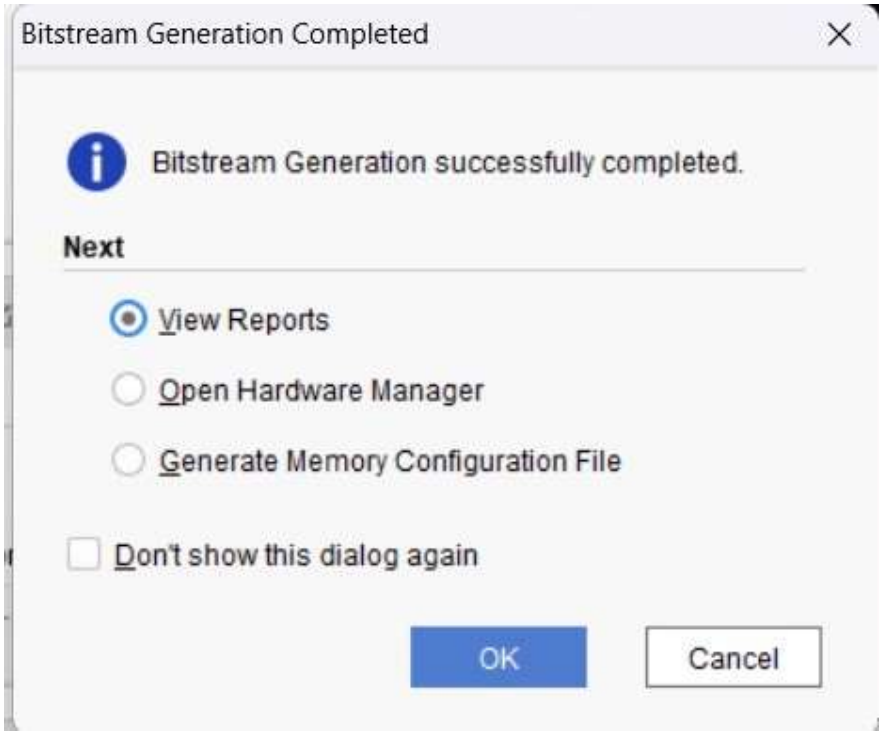


VIVADO UTILIZATION :



Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
instantiation	33	40	1	0.5	5	1
ramBlock (RAM)	2	9	1	0.5	0	0
spiBlock (SPI)	31	23	0	0	0	0

VIVADO TIMING SUMMERY :



VIVADO GENERATE BITSTRAEM :